

Documentación de ShareUrCar

Introducción y Tecnologías

Empezamos este documento explicando la base de nuestro proyecto. Para el desarrollo hemos decidido utilizar dos tecnologías que hemos visto durante el curso, usando **Spring Boot** para todo el backend y **Flutter** para la aplicación frontend. Tomamos esta decisión porque nos sentimos cómodos con ellas y nos permiten tener el código bien separado entre la interfaz que ve el usuario y la lógica del servidor.

Nombre	Fecha de modificación	Tipo	Tamaño
Documentacion	26/05/2026 20:48	Carpeta de archivos	
shareurcar_app_frontend	26/05/2026 12:42	Carpeta de archivos	
shareurcar-backend	25/05/2026 12:50	Carpeta de archivos	

Cambios que se han hecho

Durante las primeras semanas nos fuimos dando cuenta de que nuestro planteamiento inicial de la base de datos se quedaba corto para una aplicación de viajes compartidos real. Al principio pensábamos manejar las rutas de forma sencilla con nombres de ciudades, pero pronto vimos que necesitábamos guardar la latitud y longitud exacta de los orígenes y destinos. Esto nos obligó a modificar las entidades principales para poder guardar estas coordenadas y calcular distancias de forma precisa. En concreto, tuvimos que añadir las columnas **origin_lat**, **origin_lng**, **dest_lat** y **dest_lng** directamente en nuestra tabla principal de **routes**.

Otra cosa importante que tuvimos que reestructurar en la base de datos sobre la marcha fue la gestión de los pasajeros. Nos dimos cuenta de que, si un viaje tenía varios pasajeros, no podíamos guardar el estado de confirmación del viaje directamente en la ruta general porque cada persona llega o confirma en momentos distintos. Para esto teníamos dos tablas: **travel_groups**, que relaciona la ruta con el conductor y sus horarios, y **group_passengers**, que vincula a cada usuario individual con ese grupo de viaje. De esta forma, cada usuario tiene su propio estado de confirmación en la base de datos, lo que hace el sistema mucho más seguro y evita que el conductor pueda finalizar un viaje si los pasajeros reales no han confirmado su llegada. A nivel de base de datos, esto supuso modificar la tabla añadiendo la columna booleana **driver_confirmed** y la columna **status** en la tabla **routes**, además de añadir la columna **confirmed** en la tabla **group_passengers**.

También hemos querido añadir un sistema de incidencias de forma que tanto conductores como pasajeros puedan reportar problemas que se guardan directamente en el servidor. Para gestionar esto creamos la tabla entera de **incidents**, que incluye columnas clave como **id_route**, **id_reporter**, **description** y un **status** para saber si la incidencia está abierta o ya se ha resuelto.

Flujo de la aplicación

Respecto al flujo de la aplicación, el usuario interactúa con varias pestañas principales que agrupan toda esta lógica. En las pantallas de crear y buscar ruta que se accede a través del menú de inicio, implementamos una integración con los servicios de **Mapbox**. Usamos un **token** de acceso para que los usuarios puedan escribir el nombre de una calle o un centro educativo y el sistema les devuelva las sugerencias y coordenadas exactas. En esta parte del código del frontend tuvimos que investigar cómo gestionar para ocultar el token de seguridad porque al hacer commit daba un aviso. Para ello pusimos el token en un archivo aparte llamado **.env** que es donde suele meterse por lo que hemos visto, y metiéndolo en el **.gitignore** ya nos dejaba. Al final conseguimos que el mapa y las direcciones funcionen en tiempo real, y hemos puesto para las sugerencias estén centradas en las coordenadas de Alicante para que no saliesen calles de otras ciudades.

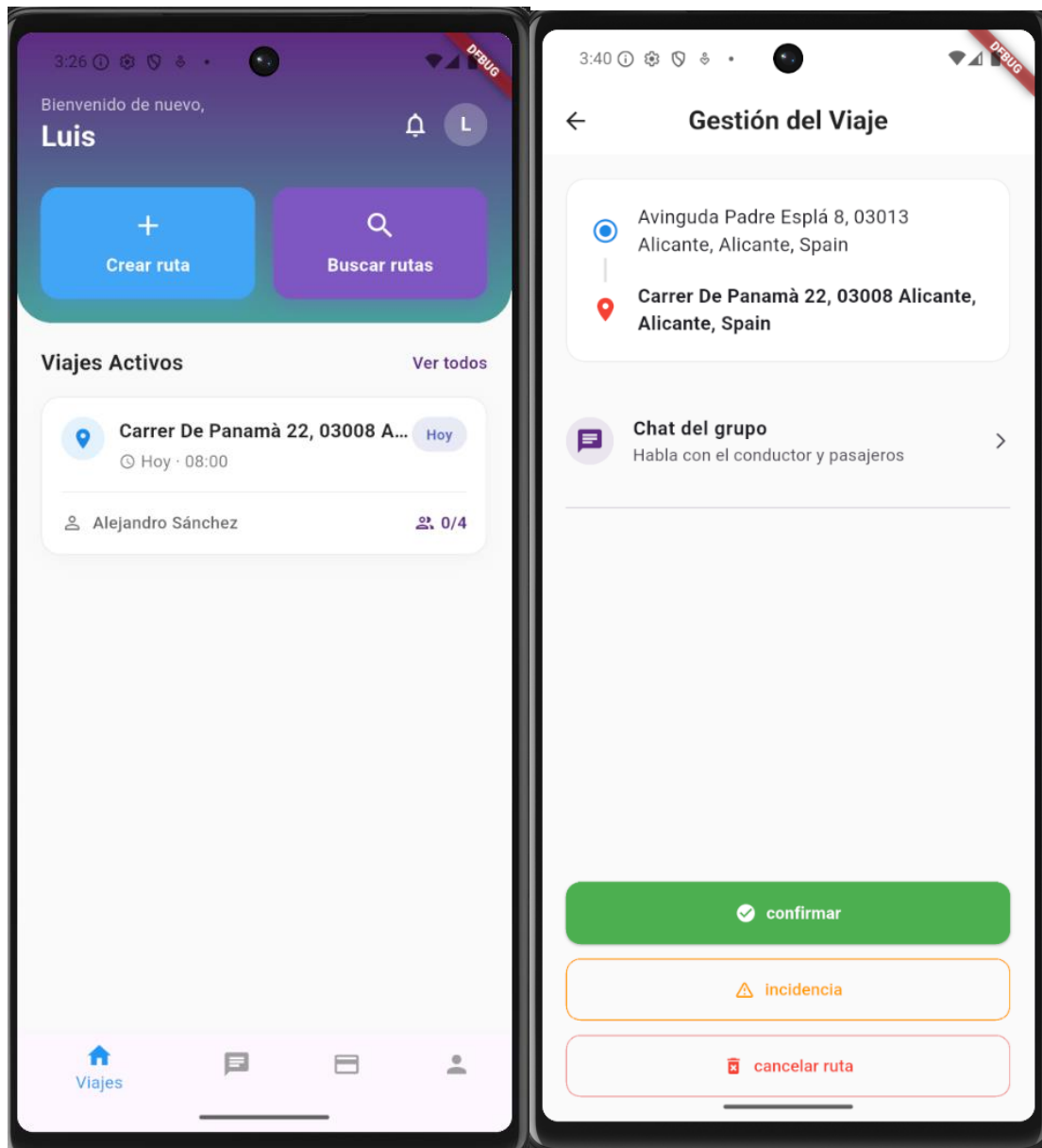
Pantalla Inicio (Viajes)

La experiencia del usuario se centra en la **pantalla de inicio**, donde tenemos los botones principales para **crear y buscar rutas**, junto con una lista de nuestros viajes activos. Al pulsar sobre la tarjeta de uno de estos viajes (es decir, uno al que ya nos hemos unido), el usuario pasa a la **pestaña detallada de gestión del viaje**. En esta pantalla aparece primero el origen y destino de la ruta, seguido de un botón para ir al chat del grupo.

Debajo encontramos los botones de acción, y aquí es una de las partes donde tuvimos que añadir columnas a la base de datos (añadiendo la columna booleana **driver_confirmed** y **status** a la tabla **routes**, y la columna **confirmed** a la tabla **group_passengers**).

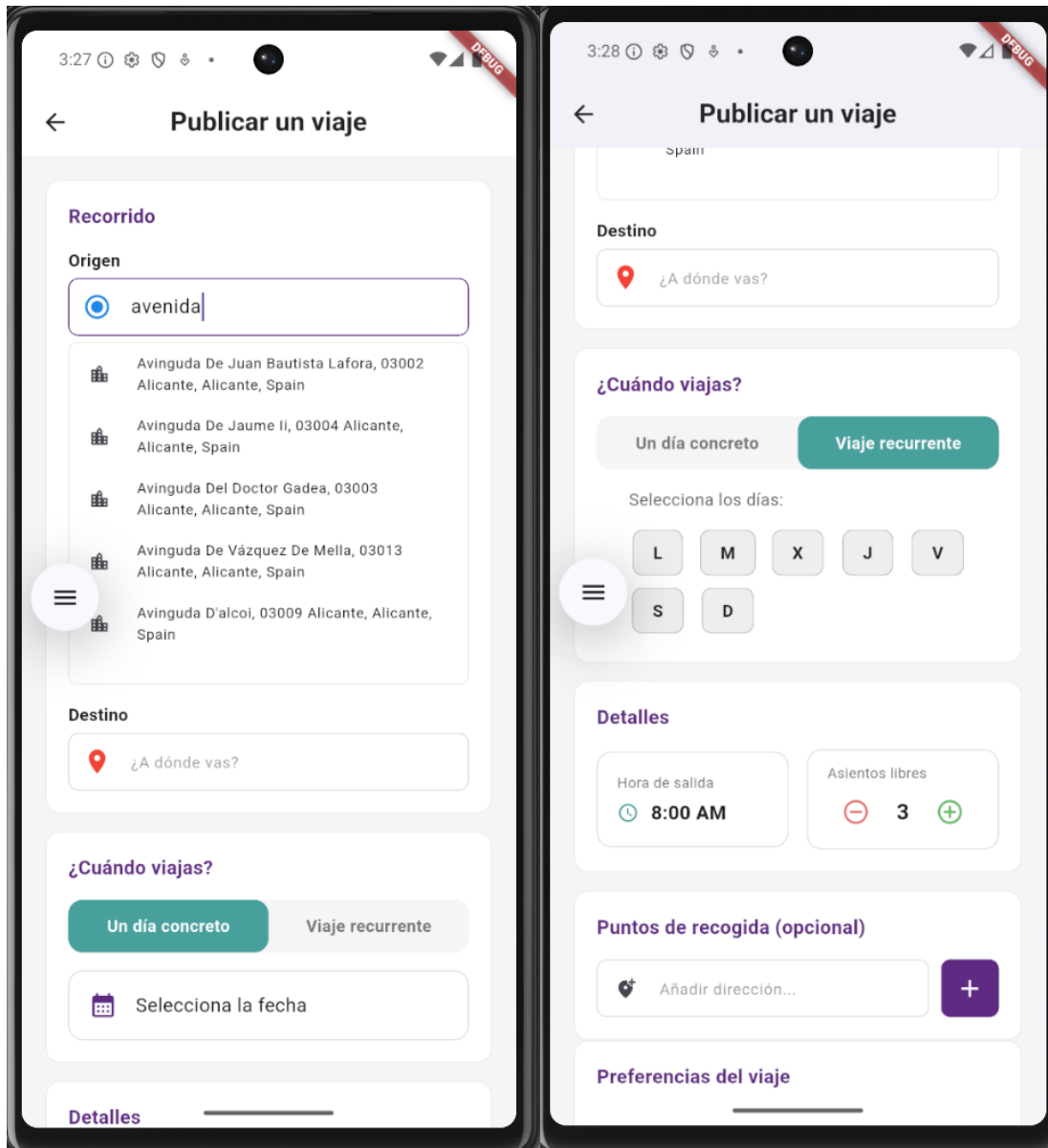
Implementamos un **sistema de doble confirmación** con el botón de "Confirmar Llegada". El viaje no se da por cerrado hasta que todos los involucrados (conductor y pasajeros) pulsan este botón. Toda esta lógica la gestionamos en los servicios del backend, comprobando en cada petición si todos los miembros del grupo han confirmado con una función privada en el service. Cuando esto ocurre, el backend marca la ruta como completada y actualiza las estadísticas del perfil del usuario (sumando +1 a los viajes completados). Además, visualmente ese viaje desaparecerá de nuestra lista de viajes activos si era de un solo día, o pasará directamente a la parte de los pagos si era una ruta semanal.

Finalmente, en esta misma pantalla de gestión tenemos un **botón de incidencias** para reportar si ha pasado algo durante el trayecto (lo cual alertaría al sistema y te excluye del pago), y un último botón rojo para **cancelar la ruta o abandonar la plaza**, el cual cambiará su texto y su función dependiendo de si tu rol en ese viaje específico es de conductor o de pasajero.

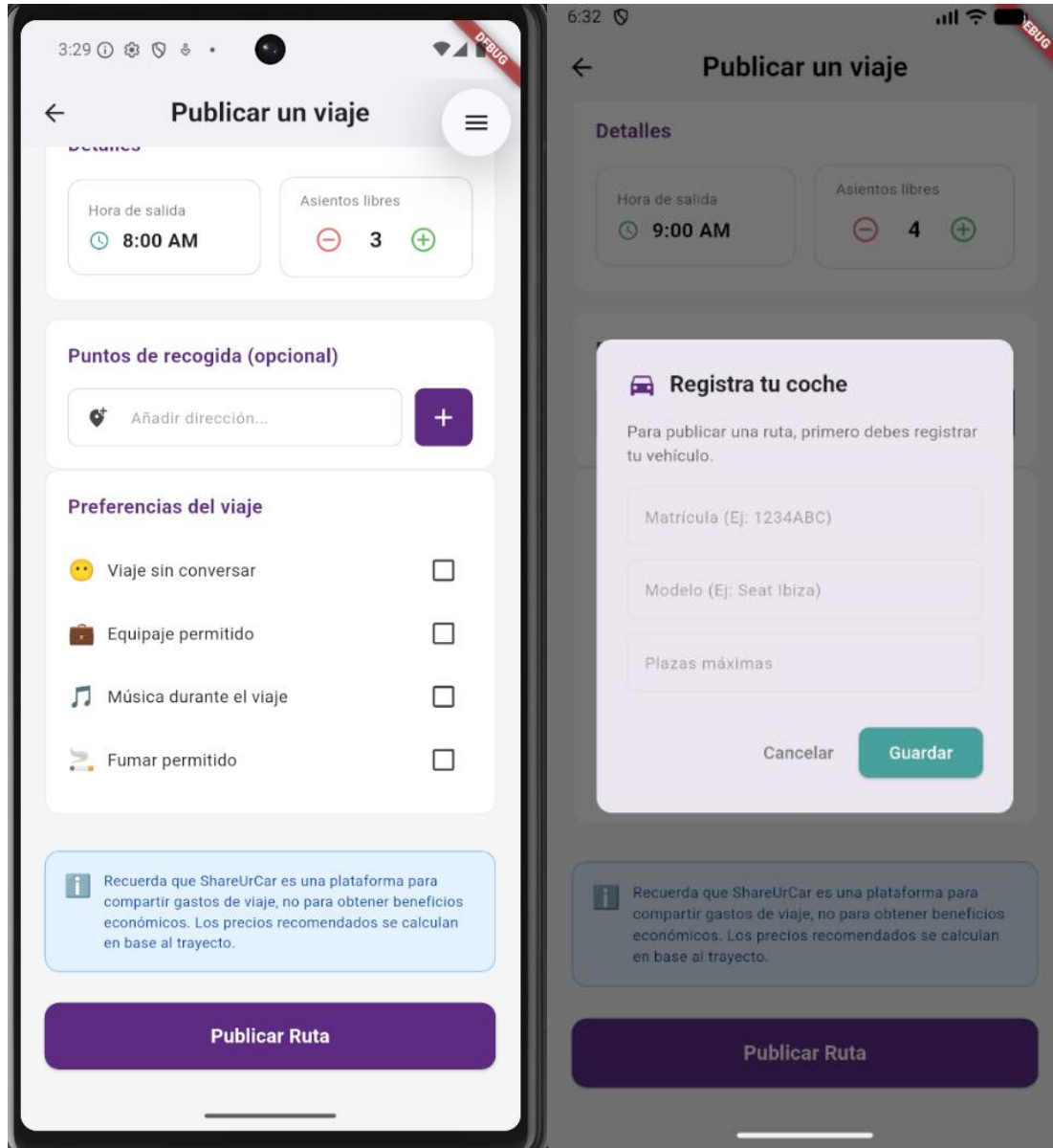


Pantalla Crear Ruta

Pantalla de crear ruta donde se muestran **sugerencias**, y el resto de la pantalla donde puede verse el sistema de elección de días de ruta:

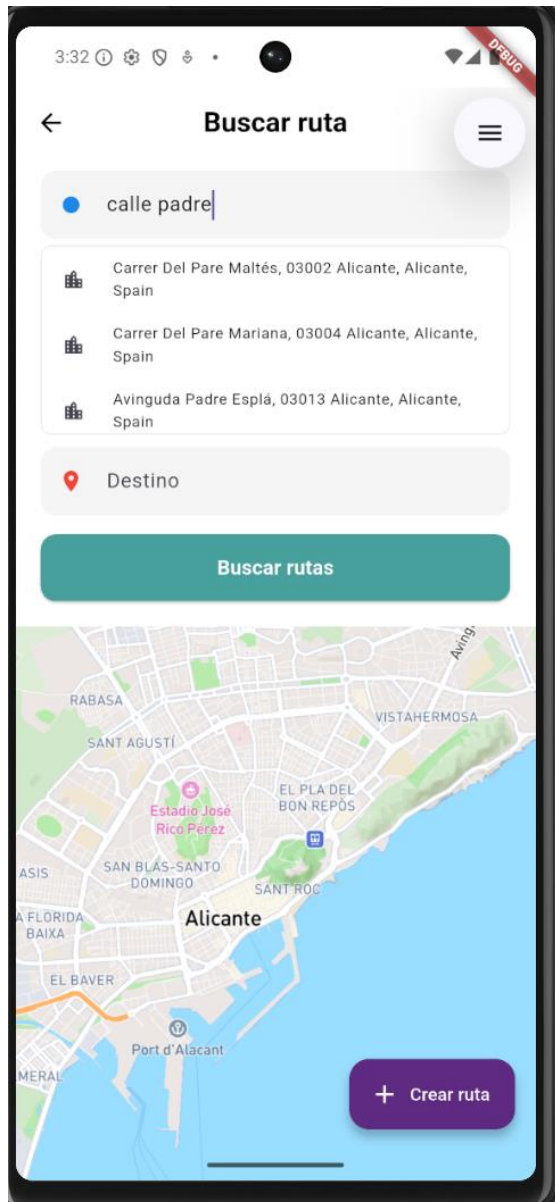


Las **preferencias, puntos de recogida y el botón de publicar ruta** se ven como en la imagen de la izquierda, y luego, el usuario al crear una ruta si NO tiene registrado un vehículo no puede crear la ruta porque todavía no se ha convertido en un **Driver**. Para facilitar esto para el usuario en el momento que le dé a publicar le saldrá una ventana para rellenar los datos de su coche, como se puede ver en la derecha:



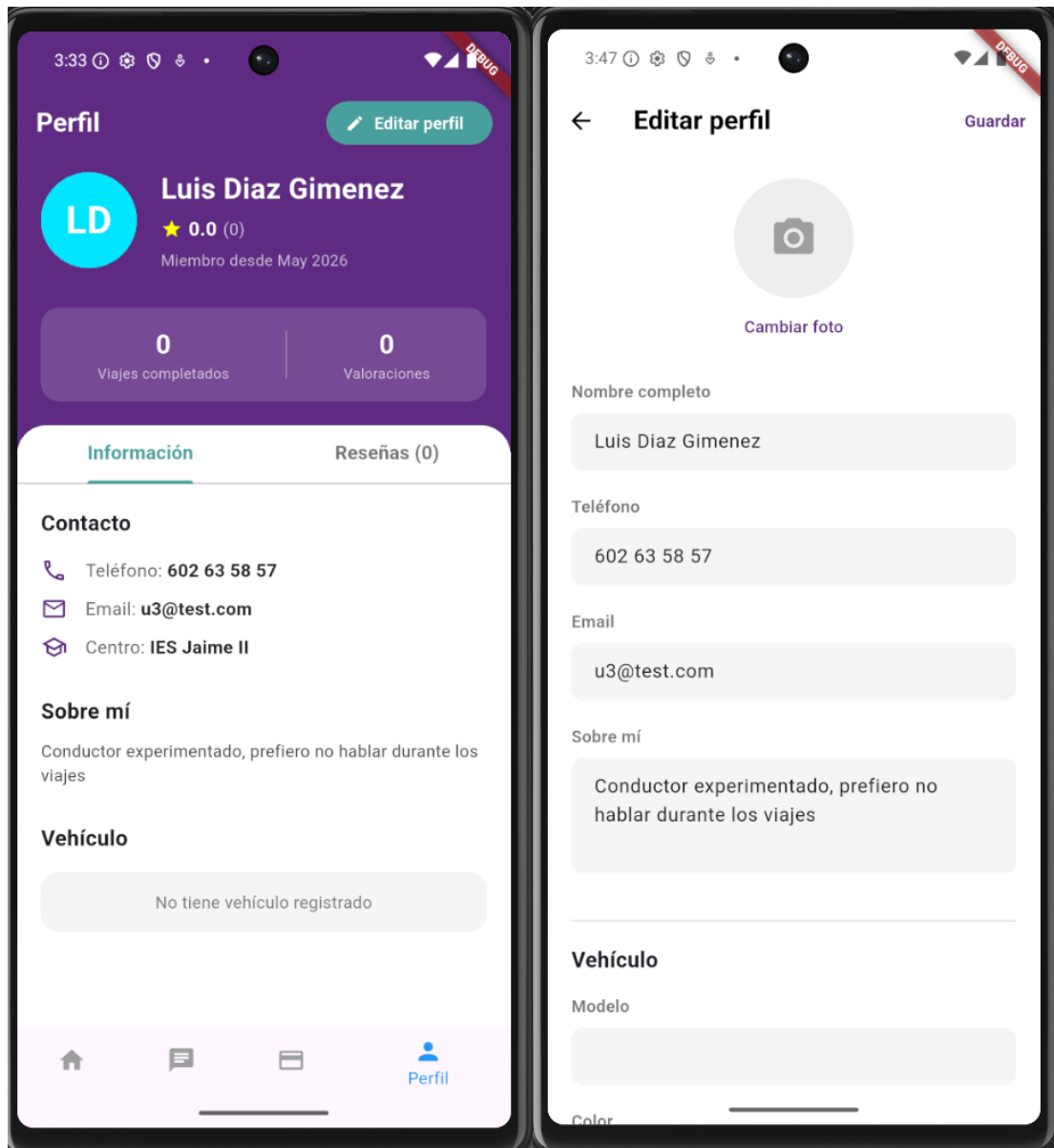
Pantalla Buscar Ruta

El **mapa interactivo de MapBox** se vería así en la pestaña de buscar rutas, donde podemos introducir origen y destino mediante las sugerencias y nos saldrán **resultados similares o exactos**. También con un botón para ir a crear ruta directamente:



Pantalla Perfil

Después tendríamos la pantalla de perfil donde se puede ver sus **viajes completados**, **reseñas**, **datos de contacto**, un espacio para que el usuario escriba sus **preferencias**, y el **vehículo registrado** (si tiene). Y también la de **editar perfil**, donde el usuario podrá **cambiar su foto** por una de la galería, **editar sus datos** y **añadir un vehículo** para poder crear rutas. Se verían así:



Sobre la parte de buscar rutas, una vez que tenemos las coordenadas reales, llegó una de las partes más complejas del código de nuestro backend. Tuvimos que buscar e implementar una función matemática bastante larga conocida como la **fórmula de Haversine**, que la usan otras aplicaciones de carpooling. Esta función, que metimos directamente en las queries de nuestro repository de rutas, se encarga de **calcular la distancia** real en línea recta entre dos

puntos **usando su latitud y longitud** (las que tuvimos que añadir):

```
@Query(value = """
    SELECT * FROM routes r
    WHERE\s
    (6371 * acos(cos(radians(:oLat)) * cos(radians(r.origin_lat))\s
    * cos(radians(r.origin_lng) - radians(:oLng))\s
    + sin(radians(:oLat)) * sin(radians(r.origin_lat)))) <= :radius
    AND\s
    (6371 * acos(cos(radians(:dLat)) * cos(radians(r.destination_lat))\s
    * cos(radians(r.destination_lng) - radians(:dLng))\s
    + sin(radians(:dLat)) * sin(radians(r.destination_lat)))) <= :radius
    ORDER BY (6371 * acos(cos(radians(:oLat)) * cos(radians(r.origin_lat))\s
    * cos(radians(r.origin_lng) - radians(:oLng))\s
    + sin(radians(:oLat)) * sin(radians(r.origin_lat)))) ASC
    \s""", nativeQuery = true)

List<Route> findNearbyRoutes(
    @Param("oLat") Double oLat,
    @Param("oLng") Double oLng,
    @Param("dLat") Double dLat,
    @Param("dLng") Double dLng,
    @Param("radius") Double radius
);
```

Gracias a esto, cuando un pasajero busca un viaje, el servidor calcula las distancias y le **devuelve solo las rutas que pasan cerca de su ubicación en un radio concreto**.

Por otro lado, nos dimos cuenta de un posible problema con las confirmaciones manuales. ¿Qué pasa si el conductor o los pasajeros se olvidan de darle al botón de confirmar cuando llegan a su destino? El viaje se quedaría colgado en estado pendiente para siempre y no se sumarían las estadísticas. Para solucionar esto buscamos y dimos con la solución (no sabíamos que se podía hacer claramente, pero lo hemos añadido para que esté más completo) de implementar una **red de seguridad automática** en el backend usando una tarea programada.

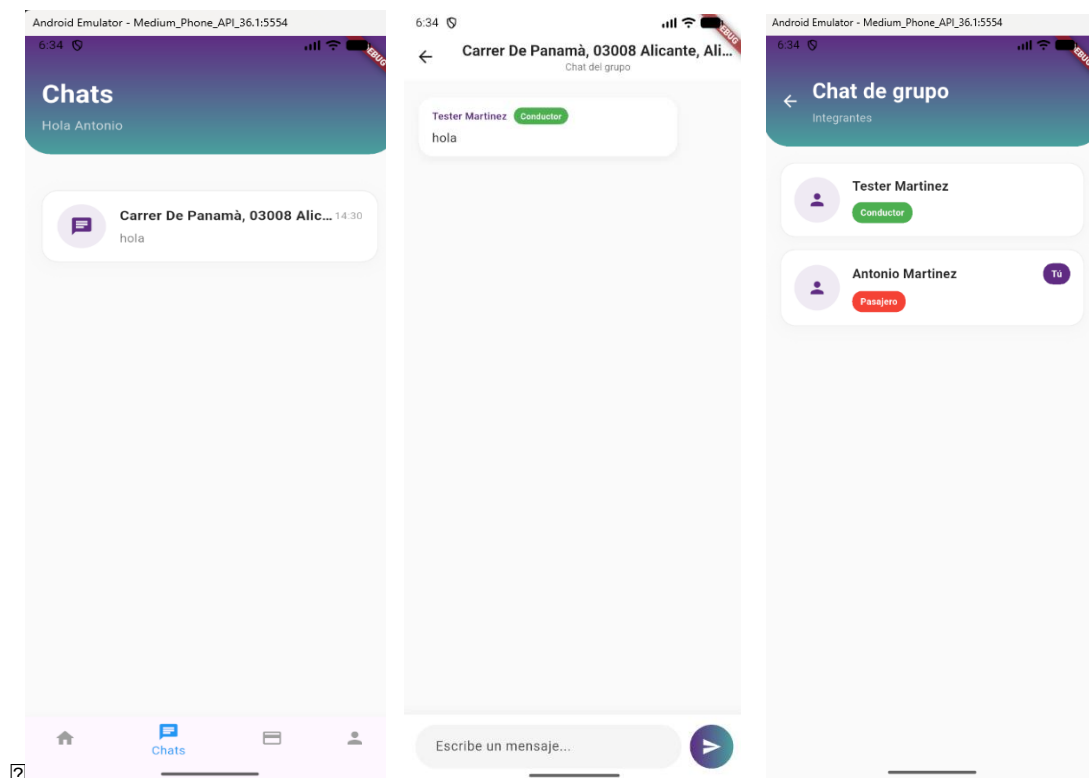
Para ello pusimos esta función en los servicios del servidor que se ejecuta por sí sola cada hora en punto. Lo que hace el código es comparar la hora actual con la base de datos y buscar todas las rutas que siguen pendientes pero cuya hora de llegada ya ha pasado. Si encuentra alguna, el sistema asume que el viaje ya ha terminado y automáticamente le cambia el estado a completado. Con esto nos aseguramos de mantener la base de datos limpia de viajes antiguos y de que el flujo de la aplicación no se rompa, aunque los usuarios sean despistados.

```
@Scheduled(cron = "0 0 * * * *") every hour Alejandro Sánchez Cordero
public void autoCompleteOldRoutes() {
    LocalDateTime now = LocalDateTime.now();
    List<Route> rutasAntiguas = routeRepository.findPendingRoutesWhereArrivalTimePassed(now);

    for (Route r : rutasAntiguas) {
        r.setStatus("COMPLETED");
        routeRepository.save(r);
    }
}
```

Pantalla Mensajes

Por otro lado, hemos añadido una pestaña dedicada a los mensajes. Esta funcionalidad permite que los usuarios que comparten un viaje puedan comunicarse entre ellos de forma privada. El sistema permite **enviar y recibir** mensajes correctamente, que desde la ventana de gestión de viajes vista anteriormente te lleva directamente al chat. Al unirse o crear una ruta se crea un chat de grupo directamente en el apartado de Chats como se ve en la primera captura. Y una vez dentro del chat si pulsas arriba sobre el nombre del chat te lleva a la última pantalla de la captura de la derecha donde puedes ver los integrantes del grupo de viaje.



Pantalla pagos

Tenemos planificado crear una pantalla principal de **pagos** donde el usuario pueda ver el resumen del **dinero que tiene que pagar por sus viajes o el dinero que va a cobrar** si es conductor. Desde ahí, podrá acceder a una pantalla secundaria diseñada para **añadir y guardar los datos de su tarjeta**. Finalmente, todo el proceso de transacción terminará en una pantalla de **aviso que le confirmará si el pago ha sido aceptado correctamente o si ha sido rechazado** por el sistema. Esta es la última funcionalidad grande que nos queda y pulir detalles.